DM585 - Online Marketplace

Mohammad Amin (moami14), Taaha Khan (tkhan23), Karim Amin (kaami23)

21/12/2025

Contents

1 Introduction

This project focuses on the design and implementation of an online marketplace application prototype developed using Flask, Jinja templates, and Docker. The application allows users to create and browse listings, communicate with other users, and leave feedback after completed transactions.

The project focuses on software architecture rather than full functionality. Initially, the system is implemented as a monolithic application to establish core features and provide a baseline design. The project then transitions the system into a microservice based architecture.

This transition highlights how architectural decisions affect system structure, communication, and data access. By comparing the monolithic and microservice based designs, differences in coupling, service responsibilities, and system complexity become more visible.

2 Application

2.1 Problem Domain

The application developed in this project is an online marketplace for buying and selling second hand items in Denmark. The problem domain addresses the growing demand for sustainable consumption, while also focusing on the challenges related to trust, transparency, and the risk of scamming in online marketplaces.

Existing Danish marketplaces such as GulogGratis primarily focus on listing volume and visibility. Although these platforms are effective and popular, trust, clear communication, and user behavior are often seen as secondary concerns. In some cases, marketplaces provide only a limited review and rating mechanism for both buyers and sellers, which can make it difficult for users to fully commit when considering buying or selling an item on the platform.

2.2 Target Users

The platform targets individuals located in Denmark who wish to sell second hand or unused items to other users, as well as browse and search for unique items across multiple categories. It is also intended for buyers who value trustworthy sellers, clear communication, and transparency when engaging in online transactions with unknown users.

The system is intended for general users of the web and does not require specialised technical knowledge, beyond basic understanding of how online marketplaces work.

2.3 High Level Functionality

The application supports a basic set of core functionalities required for a marketplace interaction, covering process from listing creation to purchase feedback.

The platform allows users to register and log in. Flask_login has been used for artefact1 to ensure that only users that are logged in can access all features, such as messaging sellers. Authenticated users can create and delete their own listings, including providing item descriptions, pricing, categorisation, location, image, and condition of the item. Listings can be searched by other users on the platform, and users can also view listings of specific categories.

To support the interaction between buyers and sellers, the application provides a simple communication mechanism that allows interested users to contact the seller of a specific listing. This enables clarification of item details, request for additional images if needed, and coordination outside the platform, such as shipping or meeting arrangements.

Additionally, the system includes a review and rating mechanism that allows users to provide feedback after purchasing an item. The seler and buyer will come to an agreement, and the buyer will receive the item. This will be arranged by the buyer and seller, and is not related to the platform. The platform servers as a means of communication for this. Once the exchange has been completed, the seller can mark the item as sold. This will give the buyer the option to review the user. The feedback consists of a 1-5 stars rating combined with a short text description of the user's experience. This functionality supports transparency and trust by enabling users to share their own experience of the other users on the platform.

Users are also provided with basic profile management features, allowing them to view their own listings and any reviews they have received. Profile related features including the ability to manage personal information, such as updating a profile picture.

2.4 User Interaction Flow

A typical user interaction flow within the application follows a simple and structured sequence.

First, a user registers an account and logs onto the platform. Once logged in, the user is presented with several marketplace options, such as browsing existing listings or selling items. If the user wishes to sell an item, they can create a new listing by providing relevant details, including description, price, category, location, image, and item condition.

When browsing listings, the user is presented with a simple filtering system that allows searching or filtering by category. When a buyer finds an item of interest, they can contact the seller through the platform's com-

munication feature to request additional information or proceed with the purchase. The buyer and seller then discuss transaction details outside the playform, such as shipping or meeting arrangements.

After the transaction between buyer and seller has taken place, the buyer can leave feedback in the form of a rating and a short review. This feedback contributes to increased transparency and helps build trust between users on the platform.

2.5 Limitations

The application is developed as a prototype marketplace system intended to demonstrate arichectural design and distributed system concepts rather than full commercial functionality.

As a result, several features commonly found in production ready online marketplaces are left out. These include integrated payment processing, more advanced search and filterings system, more advanced listing editing, offer or bidding mechanism, and more advanced rating and reputation systems. One could also argue that a "buy" option might not be necesarry, since many of the big second hand marketplaces do not have these (such as facebook marketplace).

Reviews for the seller can only be seen by the seller, but not any buyers. In the future if this marketplace was developed further, then a profile page would have to be developed for the potential buyers to see. The foundation for such a feature has already been laid, since users can leave reviews already and the reviews get stored in the database.

In addition, the user interface and user experience are kept relatively simple and functional, prioritising core functionality.

3 System architecture

3.1 Stack and framework

The backend of the application is built using Flask, a Python based web framework, that is the foundation of the system. Flask is responsible for request routing, handling user interactions, and rendering correct server side templates. Data persistence is managed using an SQLite database, accessed through the Flask SQLAlchemy. However, when running the application with Docker, one needs to use the "docker compose up –build" function to build and run the function in order to achieve data persistence. This is because using the regular docker build and run commands will create a new container with a freshly made copy of the DB, whereas the "compose" function runs the .yaml file. In the file we have specified a volume for the database, which is used for data persistence.

The application follows the application factory pattern, with the Flask application initialised in the `__init__.py` file. Funcionality is organised using Flask Blueprints, which structure the code into logical components, such as authentication, listings, user management, and user interactions.

User authentication and session management are handled using Flask_Login, while Bcrypt is used to securely hash and store user passwords. Flask_login will keep track of the user being logged in for us. Once the user has been logged in, then they can access all of the routes that have been protected with @login_required. The database stores the user information alongside the password hashes. We do not store the password itself, because if a leak were to occur, this would be detrimental to user security. User input is processed and validated using WTForms.

To support consistent development and deployment environments, the application is packaged using Docker. Docker ensures that all the dependencies are installed, which helps provide a consistent envrironment for the application to run in order to prevent issues with running it on different machines.

The user interface is implemented using site rendering Jinja HTML templates, styled with CSS and the Bootstrap framework to support responsive layout and consistent visual structure across the entire application.

3.2 Code Organization

The application follow a monolithic architcecture implemented as a single Flask application. While all functionality is within one codebase and shares common infrastructure such as the database and authentication system, the code is organised using Flask Blueprints to ensure clear seperation of functionality.

The application is initialised using a factory pattern defined in `__init__.py`, which configures core services and registers all functional blueprints before returning the configured Flask instance

Each blueprint groups related functionality into an independent module, including authentication, marketplace listings, user profile management, buyer/seller communication, and reviews.

Shared components such as form defintions, `forms.py`, and database, `models.py`, are there to ensure consistency across the application. If a part of the program wants to access one of these tables, they can easily import it and query and modify it. This is in contrast with the microservice model, where the tables are not shared outside their respective service. The presentation layer consists of Jinja templates that render dynamic HTML pages styled using Bootstrap CSS for responsive design. Static files such as CSS and default imague pictures are stored in /static/ directory

The organisation of the codebase reflects the core functionality of the ap-

plication. User can authenticate, create and browse listings, communicate with other users, and leave feedback after completed transactions. These features are supported through clearly seperated components and a structured data model, enabling consistent data handling and maintainable application flows.

Below is a diagram illustrating the monolith structure of the marketplace application, where all functionality is implemented within a single Flask application and shares a common database.

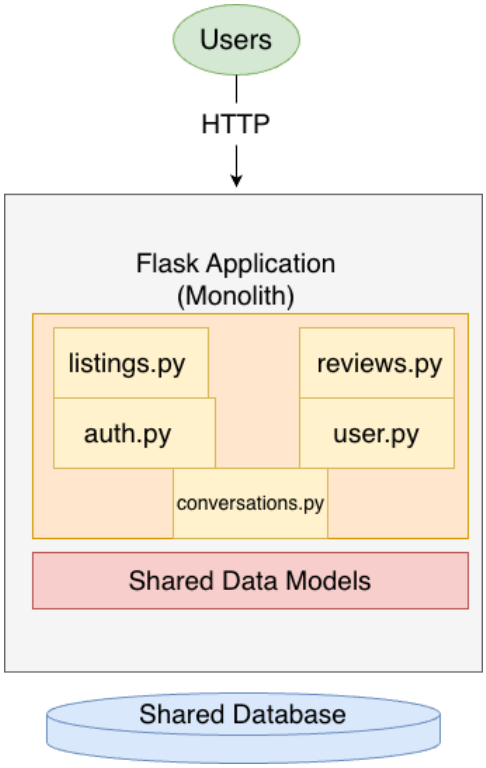

Figure 1: Monolith architecture of the marketplace application

3.3 Components

3.3.1 `auth.py`

`auth.py` is responsible for user authentication, including handling user login, registration, and logout.

Registration validates unique username and email addresses, while passwords are securely hashed using bcrypt before storage. During login, submitted information is verified against stored password hashes, and Flask Login manages authenticated user sessions once the user is successfully authenticated. Logout terminates the authenticated active session.

3.3.2 `conversations.py`

The conversation modul is responsible for the communication between buyers and sellers regarding specific listings. It enables users to initiate and participate in back and forth conversations related to marketplace items.

A conversation is automatically created once a buyer contacts a seller about a listing. All conversations relevant to the current user are stored in an inbox view, where they are accessible regardless of whether the user is acting as a buyer or a seller.

3.3.3 `forms.py`

The `forms.py` module handles form definitions and input validation for the application's core user interaction, such as authentication, listing creation, and profile management.

The module includes forms that support the main user flows, such as user registration and login, listing creation, and profile/settings updates. These forms follow validation rules such as required fields, length constraints, email form checks, and password confirmation.

3.3.4 `listings.py`

The `listings.py` module implements the core marketplace functionality related to item listings. It allows authenticated users to create listings and add item details such as name, price, condition, and optional image. This module also supports browsing, searching, and filtering available listings, as well as viewing individual item pages. Additionally, it handles deletion of a user's own listing and enables users to purchase listings they do not own.

3.3.5 `models.py`

The `models.py` defines the database models used by the application. In total, five core models are defined that represent the main data structure of the marketplace.

The models include User, which stores account and basic profile information for users of the platform, such as listings.

Listings represent items offered for sale and contain general information about the item, pricing details, and status information indicating whether an item is available or sold. Each user can have multiple listings.

Conversation represents a specific communication thread between a buyer and a seller related to a listing. One listing can have multiple conversations. Message represents the individual messages exchanged within that conversation. A conversation can contain multiple messages, form an exchange between the involvled users.

Review stores ratings and optional feedback after completed transactions and contributes to transparency and trust within the marketplace.

3.3.6 `review.py`

The `review.py` module is responsible for handling feedback after completed purchases. It allows buyers to submit a rating and an optional comment following a transaction. Each review is associated with a specific listing, where the buyer acts as the reviewer of the seller. This module supports transparency and trust by giving users the chance to share feedback based on their puchasing experience.

3.3.7 `users.py`

The `users.py` component handles user profile management and personal account settings. It allows authenticated users to view and update their profile information, including personal details and profile images.

The module also provides a user profile view where users can view the listings created by them and reviews received from other users. User actions are restricted, such that account settings can only be modified by accounts owner.

4 Microservices

Artefact2 has been made by using a microservice architecture. The system has been split into smaller, independent applications that are all run simultaneously using Docker, which allows them to communicate with each other. Each application (called service) is responsible for a specific task. Two services cannot share a database, otherwise they will be considered microservices, but rather one monolith. For our project we have made four services:

- Frontend
- Listing
- Authentication
- Reviews

The frontend service is responsible for rendering the presentation layer. The port that this service uses is also the one that the user is supposed to open in their browser. This service is responsible for rendering the HTML templates, handles user session by storing the user_id and handling user interaction. This service will communicate with the other (backend) services. For example it will make a request to the Listing service in order to fetch

the different listings that must be displayed on screen, as can be seen on the route "/". This service does not contain any business logic or database of its own.

The authentication service is responsible for keeping track of a table of users. It also manages user login, registration alongside password hashing. The frontend makes POST requests to these functions in the auth service in order to pass the user data. No other service has direct access to the data in the Users table. This isolation makes it easy to maintain, while also keeping the data secure.

The listing service keeps track of all the different listings with a database. Just like the authentication service, this service is the only one that has access to it. This service is essential, especially for future development if we are to include conversations like we did in artefact1. This is because such features would require information such as knowing the seller ID (while the buyer is logged in), and the routes like GET /listings/<int:listing_id> facilitates this.

The review service stores all the seller reviews that the buyers leaves. It operates in a similar manner, with functions that manage review creation and retrieval.

The services are loosely coupled. This is because they do not share databases, they do not call the internal function of another service, each service has its own database and logic, and all communication happens through requests. There is however some temporal coupling. This is especially present between the frontend and the other services.

- Authentication service: The frontend relies on the authentication service in order to log and register users. However, this is only necessary when the user actually performs these actions. The frontend stores the user ID with sessions, so once a user has logged in, they will remain that way even if the authentication serivce failed.

- Listing Service: The frontend relies on the listing service in order to display the different listings, which allows the user to browse. If this serice fails, then the user will still be able to use other features, like logging in, but the user would not be able to view listings or put them up. The temporal coupling is stronger in this case, but mostly because the service is used more often. The nature of the coupling is mostly the same

- Review Service: The frontend needs this service when displaying and submitting reviews. If this service is down, users can still access other features like viewing listings and logging in.

While there is some temporal coupling in the system, it is low. Only features that are directly tied to the services get affected.

In a monolithic design, all the code is run in one place. The application is one big unit. Therefore, if we need experience heavy load then we have to scale the entire application, even if its only a small part of the program that is struggling. With a microservice design we have many small and independent applications, and each can be scaled according to their needs.

The application is also easier to maintain with the microservice design, because each part is small and independent, meaning that a developer does not need to understand the entire system in order to make changes to a service. This is not the case with monolithic designs, because the different parts of the program are not separated in this fashion. During development we often changed our minds about the design of the program. For example, we originally had a "buy" button for the buyer in artefact1, however decided against this. When we made this transition, we had to check some of the other files as well for any references to this functionality. Since the application is relatively small this was not difficult, but its easy to see how it can be difficult with larger projects.

5 Transition and Reflection

The transition from a monolithic architecture to a microservice based system gave us several important trade offs. In the original monolitihc design, all components shared the same codebase and database, which simplified development, communication, and deployment initially. Communcation between components was reliable and local. However, this approach resulted in tight coupling between components, meaning that changes to one part of the system could impact unreleated functionality and required redeploying the entire application.

The idea to decompose the system into microservices focuses on high cohesion and loose coupling. Each microservice was designed around a specific business capability, such as authentication, listing, communication, or reviews, and given owernship over its own data. This aligns with the microservice principle of autonomous components that executes in isolation and interact through APIs.

A key trade off of this approach is increased architectural complexity. Functionality that was previously handled through direct function calls now requires network based communication between services. This can introduce latency and the possibility of partial failures, where one service may be unaialable while other continue to operate. The architecture therefore trades simplicity and strong consistency for improved scalability and failure isolation.

5.1 Challenges

One of the main challenges during the transition to a microservice based architecture was adapting to service to service communication. In the monolithic version, the application could directly import shared models or query the database tables. After the transition to microservices, this is no longer possible, as each service became isolated and responsible for its own data.

Instead, services had to communicate using HTTP requests and exchange data in JSON format. For example, when displaying the browse page, a request must be sent to the Listing service to retrieve the available listings. This was a significant change from the monolithic approach and required additional work to handle requests and pass the data to the user interface. This was something that took some time to get adjusted to for the group, as this method was not something we were used to. Adjusting to request based communication and losing the ability to directly import and query shared data took time to adapt to, it represents a fundamentally different development approach compared to working within a single codebase.

5.2 Architectural Quality

The final microservice based architecture improves the original monolithic design by providing a clearer functional isolation and better modularity. By isolating functionality into independent services, changes to one part of the system are less likely to affect unrelated components. This improves maintainability and support independent development and deployment of individual services.

At the same time, the arctitecture introduces more complexity compared to the monolithic design. Service to service communication, data ownership, and error handling require more careful design. Without clearly defined service boundaries and well structured APIs, services may become tangled.

5.2.1 Reflection

Due to time constraints, the microservice based system was implemented at a simplified level compared to a full prodction ready system. Some parts of the architecture were therefore kept intentionally minimal in order to focus on the core concepts and learning objectives of the project.

Because of this, not all aspects of a complete microservice architecture was implemented in detail. Instead we focused on fundamentals such as separating responsbilities between services and avoiding shared code and direct database access.

The use of service to service communication also shows how important microservice can be. Instead of relying on shared code and direct database access, service interact through clear and stable APIs.